

CNN, Block by Block

1. CNN
2. Dropout + Early Stopping
3. BatchNormalization
9. Code Review
    Training and Valid Block
    Evaluation Block

🥳 한 문장으로 비유해 보면, "CNN"은 사진을 넘겨받아 점점 **요약**해 가는 공장 라인 입니다.

- 처음엔 픽셀(28*28)을 그대로 받지만,
    - 라인을 지날수록 가로세로는 작아지고 (공간 축소)
    - 대신 "특징의 종류"는 많아집니다. (채널 증가)
- 마지막엔 "이건 숫자 몇이다" 라는 결과를 확인할 수 있습니다.

▼ 1. CNN

🗃️ `10-cnn.ipynb`

Architecture

```
class CNN(nn.Module):
    def __init__(self):
        super(CNN, self).__init__()
        self.conv1 = nn.Conv2d(1, 32, kernel_size=3, padding=1)
        self.pool = nn.MaxPool2d(kernel_size=2, stride=2)
        self.conv2 = nn.Conv2d(32, 64, kernel_size=3, padding=1)
        self.fc1 = nn.Linear(64 * 7 * 7, 128)
        self.fc2 = nn.Linear(128, 10)

    def forward(self, x):
        x = self.pool(torch.relu(self.conv1(x)))
        x = self.pool(torch.relu(self.conv2(x)))
```

```
        x = x.view(-1, 64 * 7 * 7)
        x = torch.relu(self.fc1(x))
        x = self.fc2(x)
        return x
```

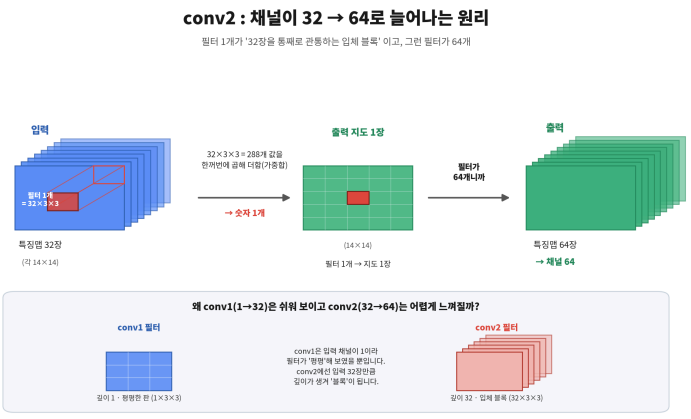
Function

def 분리해서 보기

- `__init__` : 부품을 사서 서랍에 넣어두는 단계
  - 여기에서는 아직 아무 계산도 일어나지 않음
  - `self.conv1`, `self.pool`, ... 은 "이런 도구를 준비했다"는 선언
- `forward` : 준비한 부품을 실제 순서대로 조립 및 가동하는 단계
  - 데이터 `x`가 들어와야 비로소 계산 실행
  - 위 코드에서는 순서가 곧 결과임
- `self.pool` : `__init__`에서 한번만 만들었는데, `forward`에서는 두 번 사용
  - MaxPooling은 학습되는 값이 아니라, 그냥 제일 큰 값을 고르는 규칙 → 하나의 도구를 목적에 맞게 여러 번 사용 가능
  - 반면 `conv1`, `conv2`는 각자 학습하는 필터를 가지므로 각각 정의해야 함

Channel : 1 → 32 → 64

- 흑백 MNIST 데이터셋은 채널이 1개
- `conv1`은 필터 32개를 가짐
  - 필터 하나가 이미지를 훑으면 "특정 무늬(e.g., 세로선, 곡선, 모서리)가 어디 있나"를 나타내는 지도(feature map) 한 장이 나옴
  - 필터가 32개니까 feature map 32장 ⇒ 출력 채널 32
  - `self.conv1 = nn.Conv2d(1, 32, kernel_size=3, padding=1)`
- `conv2`
  - `self.conv2 = nn.Conv2d(32, 64, kernel_size=3, padding=1)`
  - 필터 하나는 `3*3`이 아니라, `32*3*3` 입체 블록
  - 필터 하나가 32장의 feature map을 동시에 위에서 내려다 보는 것
    - 32장 각각에서 3*3씩 = 32*3*3 = 288개 숫자를 봄
    - 288개를 전부 곱해서 더함(가중합) → 숫자 하나가 나옴 → 출력 한 픽셀
    - 이 블록을 이미지 위에서부터 쭈욱 슬라이드하면 → 14*14 출력 1장



공간 크기 : 28 → 14 → 7

두 종류의 층이 크기에 다르게 작용하고 있음

- `Conv2d(..., kernel=3, padding=1)`
  - `padding=1` : 테두리를 한 겹 덧대어 크기가 그대로 유지 (28→28, 14→14)
  - 여기에서는 "크기"가 아니라 "채널"을 바꾸는 역할
- `Maxpool2d(2,2)`
  - 2*2를 1칸으로 압축하므로 크기가 절반이 됨 (28 → 14 → 7)
  - Pool은 "채널"이 아니라 "크기"를 줄이는 역할

왜 줄이면서 동시에 늘리나

- "공간"을 줄이고(pool), "특징 종류"는 늘리는(conv) 이유는
  - 어디에 있는지(위치 정보)는 점점 덜 중요해지고
  - 무엇인지(특징 정보)가 점점 더 중요해지기 때문
- 마지막에는 특징만 남아 3136(=64*7*7)개 값으로 남김

Flow

- input
  - 흑백 손글씨 이미지 1장 = (1채널, 28, 28) ← (채널, 높이, 너비)
  - 흑백 손글씨 이미지 N장 = (N장, 1채널, 28, 28) ← (batch, 채널, 높이, 너비)
  - 코드 파일 에서는 batch_size = 32

단계	코드	(C * H * W)	(N * C * H * W)	작업 내용
입력	`x`	1 * 28 * 28	N * 1 * 28 * 28	원본 픽셀
conv1	`conv1(x)`	32 * 28 * 28	N * 32 * 28 * 28	필터 32개로 feature map 32장 추출 (단, 크기 유지 ← padding)
relu	`relu(...)`	32 * 28 * 28	N * 32 * 28 * 28	음수를 0으로 (비선형성 부여)
pool	`pool(...)`	32 * 14 * 14	N * 32 * 14 * 14	2*2 최대값 → 크기 절반
conv2	`conv2(x)`	64 * 14 * 14	N * 64 * 14 * 14	feature map 32 → 64로 확장
relu	`relu(...)`	64 * 14 * 14	N * 64 * 14 * 14	음수 제거
pool	`pool(...)`	64 * 7 * 7	N * 64 * 7 * 7	크기 절반
flatten	`x.view(-1, 64*7*7)`	3136	N * 3136	3차원(64*7*7)을 1차원으로 펼침
fc1	`fc1(x)`	128	N * 128	3136개 특징을 128개로 압축/결합
relu	`relu(...)`	128	N * 128	음수 제거
fc2	`fc2(x)`	10	N * 10	정답 라벨(10개) 각각에 대한 점수 10개

FAQ

- `x.view(-1, 64*7*7)`에서 `-1`은 어떤 의미인가요?
  - `view`는 원소 개수를 그대로 둔 채 모양만 바꾸는 함수
  - 실행 전 `x`는 `(batch, 64, 7, 7)`이고, 우리는 1개의 배치 안에서 1차원 한 줄로 펼치고 싶음

    - 64*7*7 = 3136 : “한 장”을 1차원으로 펼친 길이 ⇒ `(1, 3136)`
    - 한 장이 아니라 batch 개수만큼 감안하여 처리하기 위해 `-1`로 자동으로 처리되도록 유도 ⇒ `(N, 3136)`
- 왜 `relu`를 자꾸 끼워넣나요?
    - relu가 없으면 아무리 conv 레이어를 쌓아도 수학적으로 “직선 하나”와 같아져서 복잡한 곡선(숫자 모양)을 구분 못함
    - relu는 “음수를 버린다”는 간단한 꺾임을 넣어 모델이 비선형 패턴을 배울 수 있도록 함
- conv는 크기를 안 바꾸는데 왜 있는 건가요?
    - conv의 임무는 크기 축소가 아니라 특징 추출(채널 생성)
    - 크기를 줄이는 역할은 pool이 담당
- 마지막 출력 10개는 확률값인가요?
    - 확률값으로 계산하기 전 버전인 점수(logit)
- 학습되는 값(가중치)는 어디에 있나요?
    - `conv1`, `conv2`의 필터와 `fc1`, `fc2`의 연결값에만 있음
    - `pool`과 `relu`는 학습이 아닌 처리 규칙

▼ 2. Dropout + Early Stopping

🗃️ `11-cnn-dropout-earlystopping.ipynb`

Architecture

```
class CNN(nn.Module):
    def __init__(self, dropout_p=0.5):
        super(CNN, self).__init__()
        self.conv1 = nn.Conv2d(1, 32, kernel_size=3, padding=1)
        self.pool = nn.MaxPool2d(kernel_size=2, stride=2)
        self.conv2 = nn.Conv2d(32, 64, kernel_size=3, padding=1)
        self.dropout_conv = nn.Dropout(0.25)        # (추가) conv 블록용
        self.fc1 = nn.Linear(64 * 7 * 7, 128)
        self.dropout_fc = nn.Dropout(dropout_p)     # (추가) fc 블록용, 기본 0.5
        self.fc2 = nn.Linear(128, 10)

    def forward(self, x):
        x = self.pool(torch.relu(self.conv1(x)))
        x = self.pool(torch.relu(self.conv2(x)))
        x = self.dropout_conv(x)                    # (추가)
        x = x.view(-1, 64 * 7 * 7)
        x = torch.relu(self.fc1(x))
        x = self.dropout_fc(x)                      # (추가)
        x = self.fc2(x)
        return x
```

Function

Dropout

- 10번 파일은 학습 데이터는 점점 잘 맞지만 valid loss가 정체되는 과적합 초기 신호를 보였음



- Dropout은 학습 중 뉴런 일부를 랜덤하게 0으로 꺼버림
  - 매 스텝마다 꺼지는 조합이 달라지므로, 모델은 특정 뉴런 하나에 의존할 수 없고 여러 뉴런이 골고루 특징을 배우게 됨
  - `nn.Dropout(0.25)` = 25%를 끔 ← conv
  - `nn.Dropout(0.50)` = 50%를 끔 ← fully-connected
  - FC는 연결이 압도적으로 많아지는 특징이 있어 과적합 위험을 크게 누르고자 조금 더 큰 값으로 설계
- `model.train()` 모드에서만 작동하도록 반영

Early Stopping

```
for epoch in range(num_epochs):
    # ---- Training ----
    model.train()
    running_loss = 0.0

    for inputs, labels in train_loader:
        ...

    train_losses.append(running_loss / len(train_loader))

    # ---- Validation ----
    model.eval()
    with torch.no_grad():
        for inputs, labels in valid_loader:
            inputs, labels = inputs.to(device), labels.to(device)
            outputs = model(inputs)
            loss = criterion(outputs, labels)

            valid_running_loss += loss.item()

            _, predicted = torch.max(outputs, 1)
            valid_total += labels.size(0)
            valid_correct += (predicted == labels).sum().item()

        valid_losses.append(valid_running_loss / len(valid_loader))

    # ---- Early Stopping ----
    if valid_losses[-1] < best_valid_loss:
        best_valid_loss = valid_losses[-1]
        best_model_state = copy.deepcopy(model.state_dict())
```

```
            patience_counter = 0
        else:
            patience_counter += 1
            if patience_counter >= patience:
                print(
                    f"Early stopping at epoch {epoch + 1} "
                    f"(best valid loss: {best_valid_loss:.4f})"
                )
                break

    # 가장 valid loss가 낮았던 시점의 가중치로 복원
    if best_model_state is not None:
        model.load_state_dict(best_model_state)
```

- 동작 방식 :
  - 매 epoch 마다 valid loss가 지금까지 최고 기록(`best_valid_loss`) 보다 나아졌는지 확인
    - 나아지면 → 기록 갱신 & 현재 가중치를 `best_model_state` 에 저장 & 대기 카운터 0으로 초기화
    - 안 나아지면 → 대기 카운터 +1
  - 카운터가 `patience` 도달하면 → epoch을 다 채우기 전에 중단(`break`)
    - `patience=3` : 우연히 한 두번 정체될 수 있지만, 3번 연속 개선이 없으면 학습 중단
  - 학습 종료 후 → 저장해 둔 `best_model_state`로 가중치 복원
    - `model.state_dict()`
      - 모델 안의 학습된 모든 값(conv 필터, fc 가중치, BatchNorm 값 등)을 "이름: 값" 형태로 모아 놓은 것
      - 즉 지금 학습된 모델의 스냅샷
      - (참고) pool, relu처럼 학습하지 않는 부분은 보관되지 않음
    - `copy.deepcopy(...)` - 그대로 복사해서 얼려두기
      - `best_model_state = model.state_dict()` 만 하면 → 모델 스냅샵의 바로가기 링크 설정
      - `copy.deepcopy(...)` : 완전히 분리된 독립 복사본을 만드는 것
        - 학습 내용이 바뀌어 그때그때 스냅샷이 바뀌어도
        - 특정 시점의 스냅샷은 독립되어 얼어붙은 상태로 확인 가능

FAQ

- Dropout을 넣으면 값이 사라지는데 정보 손실 아닌가요?
  - 학습 중에만 임시로 끄는 것이고, 평가 때는 모두 켜짐
  - 일부러 힘든 조건에서 학습시킨 후에, 실전에서는 온전한 실력을 사용하도록 한다는 접근
- Dropout 비율은 높을수록 좋나요?
  - 너무 높으면 배울 뉴런이 부족해져서 과소적합으로 넘어가게 됨 → train 정확도까지 하락
  - 보통 0.2 ~ 0.5 사이에서 조절
- Early Stopping을 적용하면 `num_epochs`는 의미 없나요?
  - `num_epochs`는 학습 최대 한도
  - 그 전에 개선이 멈추면 일찍 끝나고, 끝까지 개선되면 한도까지 모두 사용
- `patience`를 너무 크게/작게 잡으면 어떻게 되나요?
  - 작으면 잠깐의 정체에도 학습이 성급하게 멈춰버려 학습이 덜 됨
  - 크면 이미 과적합이 시작되었지만 계속 학습이 진행되게 됨

▼ 3. BatchNormalization

🗃️ `12-cnn-batchnorm.ipynb`

BatchNorm은 마치 각 공정(conv, fc 등) 사이에 놓인 "검수대" 같은 역할 입니다.

다음 공정에 넘기기 전에 재료를 항상 "표준 규격"으로 맞춰주는 것으로, 뒷 공정은 늘 일정한 분포의 입력을 받을 수 있게 됩니다.

Architecture

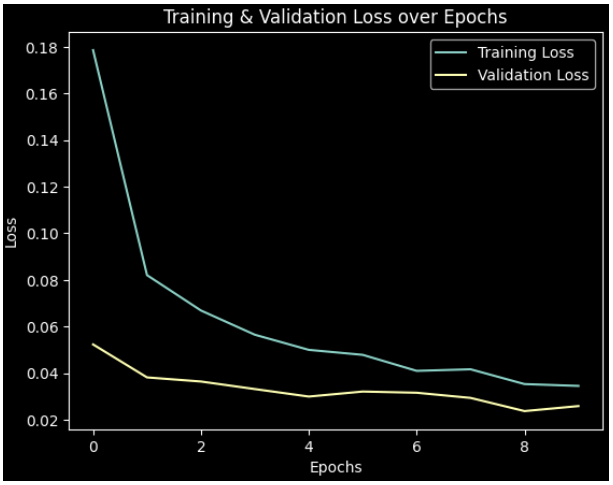
```
class CNN(nn.Module):
    def __init__(self, dropout_p=0.5):
        super(CNN, self).__init__()
        self.conv1 = nn.Conv2d(1, 32, kernel_size=3, padding=1)
        self.bn1 = nn.BatchNorm2d(32)        # (추가) conv1 출력 정규화
        self.pool = nn.MaxPool2d(kernel_size=2, stride=2)
        self.conv2 = nn.Conv2d(32, 64, kernel_size=3, padding=1)
        self.bn2 = nn.BatchNorm2d(64)        # (추가) conv2 출력 정규화
        self.dropout_conv = nn.Dropout(0.25)
        self.fc1 = nn.Linear(64 * 7 * 7, 128)
        self.bn_fc = nn.BatchNorm1d(128)     # (추가) fc1 출력 정규화
        self.dropout_fc = nn.Dropout(dropout_p)
        self.fc2 = nn.Linear(128, 10)

    def forward(self, x):
        # 순서 주의: Conv → BatchNorm → ReLU → Pool
        x = self.pool(torch.relu(self.bn1(self.conv1(x))))
        x = self.pool(torch.relu(self.bn2(self.conv2(x))))
        x = self.dropout_conv(x)
        x = x.view(-1, 64 * 7 * 7)
        x = torch.relu(self.bn_fc(self.fc1(x)))
        x = self.dropout_fc(x)
        x = self.fc2(x)
        return x
```

Function

- BatchNorm2d vs BatchNorm1d
  - 하는 역할은 같고, "무엇을 하나의 규격 단위로 볼지"만 다름
  - BatchNorm2d
    - conv 출력이 아직 이미지 묶음 형태, `(N, C, H, W)` = (이미지 장수, 채널, 높이, 너비)
    - conv1 출력 → `(32, 32, 28, 28)`
    - 규격 단위 = 채널 하나 → conv1 채널이 32개 → 검수 규격 32개
  - BatchNorm1d
    - fc 출력은 펼쳐진 숫자 목록, `(N, 특징수)` = (샘플수, 뉴런개수)
    - fc1 출력 → `(32, 128)`
    - 규격 단위 = 뉴런 하나 → fc1 뉴런 128개 → 검수 규격 128개
- 순서가 중요 : conv → BN → ReLU
  - 정규화는 ReLU 이전에 적용함
  - ReLU가 음수를 잘라내기 전에 분포를 먼저 다듬어야 검수대 역할을 제대로 할 수 있음

- train, eval에서 각각 다르게 동작
  - train : 각 배치의 평균/분산을 그때그때 계산해 사용, 동시에 전체 이동 평균을 누적해 둠
  - eval : 배치가 아니라 학습 때 누적해 둔 고정 통계값 사용
  - valid loss가 train loss 보다 낮게 나오는 구간이 확인되는데, 이는 일반화 관점이 아니라 계산 방식의 차이가 반영된 것으로 해석하는 것이 적절



FAQ

- BatchNorm과 Dropout은 둘 다 규제인데 같이 써도 되나요?
  - 모두 적용해도 됩니다
  - 다만 앞서 확인한 train/eval 계산 차이 때문에 손실 곡선 모양이 달라질 수 있으니, 수치 비교는 eval에서 하는 것이 적절
- 데이터를 입력할 때 이미 정규화가 적용되었는데, BatchNorm을 추가로 적용해도 되나요?
  - 네. 가능합니다. 두 정규화는 다른 목적으로 작동합니다.
  - 입력 정규화 (transform)
    - `ToTensor()`로 [0,1] 범위가 된 픽셀을 `Normalize((0.5, ), (0.5, ))` 가 [-1, 1]로 맵핑
    - 고정된 mean/std로 원본 입력을 한번만 적용
    - 첫번째 레이어가 받는 입력의 스케일을 잘 맞추어 초기 학습이 안정적으로 시작되도록 하는 것 → 입력층 앞단에만 영향
  - BatchNorm
    - conv 레이어를 거치면서 활성값의 분포는 가중치가 바뀔 때마다 계속 이동 → 입력 정규화는 처리할 수 없음
    - BatchNorm은 각 레이어 지점에서 이 분포를 다시 정규화하여 손실 지형을 매끄럽게 만드는 효과
- 배치 크기가 너무 작으면 문제가 되나요?
  - 배치가 아주 작으면 배치의 평균과 분산이 들쭉날쭉해져 정규화가 불안정해질 수 있음
- BatchNorm을 넣으면 항상 성능이 개선되나요?
  - 꼭 그렇지 않습니다
  - 1회 실행만으로 개선 폭을 단정하기 어렵고, 반복 실행에서 같은 경향이 유지되는지 확인하는 것이 필요

9. Code Review

▼ Training and Valid Block

🗃️ `10-cnn.ipynb` > 21번 셀

정리할 3가지 :

- batch : 데이터를 한 번에 다 넣지 않고 32장씩 묶어서 넣음. `train_loader`는 이 묶음들을 하나씩 꺼내 주는 역할 담당
- epoch : 전체 데이터를 한 바퀴 다 도는 것. 모든 batch를 한번씩 처리하면 1 epoch.
- loss : batch 하나를 처리할 때마다 "이 묶음에서 얼마나 틀렸나"가 숫자 하나로 나옴

Training

```
for epoch in range(num_epochs):
    # ---- Training ----
    model.train()
    running_loss = 0.0

    for inputs, labels in train_loader:
        inputs, labels = inputs.to(device), labels.to(device)

        optimizer.zero_grad()
        outputs = model(inputs)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()

        running_loss += loss.item()

    train_losses.append(running_loss / len(train_loader))
```

- `running_loss += loss.item()`
  - 한 epoch 안에는 batch가 아주 많음 (e.g., train 54,000장 / 32 = 약 1,688개 batch)
  - batch를 처리할 때마다 loss가 각각 계산되는데, 우리가 필요한 것은 각각이 아니라 "이번 epoch 전체에서 평균적으로 얼마나 틀렸나" 임
  - 이를 위해 batch 마다 나오는 loss를 `running_loss`에 계속 더해서 1개 epoch 내의 총합을 먼저 모음
    - `+=` : 원래 값에 더해서 다시 저장해라
    - `running_loss = 0.0` 에서 시작해서 batch가 돌 때마다 점점 쌓임
    - (비유) 매 수업 후 쪽지시험 점수를 계속 합산하여, 한 학기의 총 점수를 만드는 것
  - `.item()` :
    - `loss`는 단순 숫자가 아니라 계산 과정 정보까지 담은 tensor
    - 우리는 그 안에서 loss 숫자만 필요하기 때문에 `.item()`으로 꺼냄
    - 이를 사용하지 않으면 불필요한 계산 정보가 계속 매달려 메모리를 낭비하게 됨
- `train_losses.append(running_loss / len(train_loader))`
  - 단위 : epoch
  - `running_loss`는 총합이라, batch가 많을수록 무조건 커짐. 그래서 총합 그대로는 epoch끼리 비교할 수 없음
    - `len(train_loader)` = batch 개수
    - loss 총합을 batch 개수로 나누면 → batch당 평균 loss → 이번 epoch을 대표하는 숫자 하나로 표현
  - `.append`는 평균값을 `train_losses` 리스트에 한 개씩 쌓는 것
    - epoch을 10번 돌면 리스트에 값 10개가 순서대로 쌓임

    - 이렇게 모아 둬야 나중에 `plt.plot(train_losses)`로 epoch별 학습 곡선을 그려 확인할 수 있음

Valid

```
for epoch in range(num_epochs):
    # ---- Validation ----
    model.eval()
    valid_running_loss = 0.0
    valid_correct = 0
    valid_total = 0

    with torch.no_grad():
        for inputs, labels in valid_loader:
            inputs, labels = inputs.to(device), labels.to(device)
            outputs = model(inputs)
            loss = criterion(outputs, labels)

            valid_running_loss += loss.item()

            _, predicted = torch.max(outputs, 1)
            valid_total += labels.size(0)
            valid_correct += (predicted == labels).sum().item()

    valid_losses.append(valid_running_loss / len(valid_loader))
    valid_accuracies.append(valid_correct / valid_total)
```

- `valid_running_loss += loss.item()` → `valid_losses.append(valid_running_loss / len(valid_loader))`
    - Training과 같은 흐름 : batch loss 누적 → 평균 → 기록
    - 단, valid는 채점만 하므로 `model.eval()` + `torch.no_grad()` 안에서 실행
        - `torch.no_grad()` : 가중치를 바꾸지 않음
- Training에 없는 정확도 추가
    - `valid_correct += (predicted == labels).sum().item()` : 맞힌 개수 누적
    - `valid_total += labels.size(0)` : 전체 개수 누적
    - `valid_accuracies.append(valid_correct / valid_total)` : 맞힌 개수 / 전체 개수 = 정확도

FAQ

- `running_loss = 0.0`은 왜 for문 안 (epoch 마다)에 있나요?
    - epoch 마다 총점을 새로 내야 하기 때문
    - 밖에 두면 지난 epoch 점수까지 섞여버리기 때문
- `len(train_loader)`는 데이터 개수 인가요? batch 개수 인가요?
    - batch 개수
    - 참고로 데이터 개수는 `len(train_loader.dataset)`
- `train_losses`에는 값이 몇 개 담기나요?
    - epoch 당 1개
    - epcoh이 10번 돌면 길이 10개인 리스트가 됨
- 그냥 `print`만 하면 안되고 꼭 `append` 해야 하나요?
    - 화면에 찍히지만 값은 사라짐
    - 그래프로 추이 확인, 서비스 모니터링 등을 위해서는 리스트에 저장해 둬야 함

- 왜 총합을 모았다가 나누나요? 처음부터 평균은 안 되나요?
  - loss가 batch 마다 하나씩 순서대로 나오기 때문
  - 일단 더해 두었다가 마지막에 개수로 한 번 나누는 방식이 가장 간단하고 메모리 절약적

▼ Evaluation Block

🗃️ `10-cnn.ipynb` > 22번 셀

이제 학습된 모델은...

- 이미지 한 장마다 10개의 점수(logit)을 내놓음 : 0~9 라벨 각각에 대한 점수
- 그 중 가장 높은 점수의 번호가 곧 모델의 답
- 해당 결과로 정확도(accuracy) 확인 가능

Evaluation 함수

```
def evaluate(model, loader, collect_misclassified=False):
    model.eval()                        # ① "지금은 시험 시간" 선언
    correct = 0
    total = 0
    misclassified = []

    with torch.no_grad():               # ② 채점만, 공부(기울기 계산)는 안 함
        for inputs, labels in loader:
            inputs, labels = inputs.to(device), labels.to(device)
            outputs = model(inputs)

            _, predicted = torch.max(outputs, 1)   # ③ 가장 높은 점수의 번호 = 예측

            total += labels.size(0)                          # ④ 전체 개수 누적
            correct += (predicted == labels).sum().item()    # ④ 맞힌 개수 누적

            if collect_misclassified:                        # ⑤ (선택) 틀린 것 모으
기
                wrong_idx = (predicted != labels).nonzero(as_tuple=True)[0]
                for idx in wrong_idx:
                    misclassified.append(
                        (inputs[idx].cpu(), predicted[idx].cpu(), labels[idx].cpu
())
                    )

    accuracy = correct / total          # ⑥ 정답률 = 맞힌 수 ÷ 전체 수
    return accuracy, misclassified      # ⑦ 점수(와 틀린 목록)를 돌려줌
```

- ① `model.eval()` - "지금은 시험 시간"이라고 선언
  - 모델에게 지금부터 평가 중임을 알려주는 스위치 역할
  - 이를 통해 모드 별 적용이 달라지게 됨 - Dropout은 뉴런을 모두 켜고, BatchNorm은 누적해 둔 고정 통계값을 사용
- ② `with torch.no_grad()` - 학습 없음
  - Training 때는 어디를 얼마나 고쳐야 하는지를 계산하려고 기울기 추적
  - Evaluation은 고칠 것이 없으니 추적도 필요하지 않음 → `torch.no_grad()`는 기울기 추적을 끄라는 선언
- ③ `torch.max(outputs, 1)` - 예측 결과
  - `outputs` : 이미지 한 장당 10개 점수

	위치0	위치1	위치2	…	결과
이미지 A	0.1	2.5	0.3		위치1
이미지 B	3.1	0.2	0.4		위치0
이미지 C	0.5	0.6	4.2		위치2
…					

    - `torch.max` : (가장 큰 값, 그 위치값) 두 개를 반환 → 몇 번째 위치가 답인가 = 어떤 라벨로 예측 되었나
        - `_, predicted` = (점수, 위치값) : 점수는 버리고, 위치값을 `predicted`에 할당
        - `1` :
            - PyTorch는 표의 방향에 번호를 붙임 : 0 = 세로 방향, 1 = 가로 방향
            - 즉, outputs의 가로 방향으로 최대값을 가진 위치 확인
- ④ 정확도 계산
    - `total += labels.size(0)` : 이번 배치의 이미지 개수를 전체 개수에 더함 → `labels.size(0)` = 해당 묶음 안의 이미지 수
    - `correct += (predicted == labels).sum().item()` : 맞힌 개수 누적
        - `predicted == labels` : 위치마다 맞으면 True, 틀리면 False
        - `.sum()` : True 개수 → `.item()` : 순수 숫자만
- ⑤ 틀린 문제 발라내기 (Optional)
    - `collect_misclassified=False` : 기본값(False)면 점수만 계산. `True`로 선언하면 틀린 샘플만 별도로 수집
    - `(predicted != labels).nonzero()` : 예측이 틀린 위치를 찾음
    - 찾은 위치의 (입력 데이터, 예측값, 실제값)을 `misclassified` 목록에 담음
        - `.cpu()` : 나중에 그림으로 확인하려면 데이터를 CPU로 가져와야 함 (matplotlib는 CPU 데이터만 가능)

FAQ

- `model.eval()`을 안 쓰면 어떻게 되나요?
    - Dropout, BatchNorm이 적용되면 평가값이 왜곡됨
    - 항상 켜두는 습관이 안전함
- `torch.no_grad()`를 빼면 결과가 틀리나요?
    - 결과는 틀리지 않음
    - 다만 불필요하게 느려지고 메모리 사용량 증가
- `correct`, `total`은 왜 함수 맨 위에서 0으로 시작하나요?
    - 이번 채점을 새롭게 세기 위해 맨 앞에서 선언
    - loader를 다 돌면서 누적한 뒤 마지막에 한번 나눔